后训练学习计划 • 第 6 周（Phase 2 开工 • 10/12-10/18 • SFT 全流程 + PEFT，M2 起步）

本周定位：**分界线。前四周你在买「懂原理」，从这周开始你要卖「会调训练」。** 主题是把一个只会续写的 base 模型用指令数据变成会对话的模型，并搞清岗位真正会考的三件事：chat template、loss masking、LoRA/QLoRA 的取舍。交付物是 `m2-sft`（LoRA adapter + 配置 + 前后对比）、一份冻结的 20 题评测表、闭卷默写的 SFT loss 与 mask 规则。 用法说明：每天先看「今日总览」，再从任务 1 逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗 = 硬性要求**，灰色小字块 = 注释（可跳过，不影响主线）。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一 10/12	阶段切换 + 清周 5 gaps + base vs instruct 实测 + chat template	对比表 5 行 + template 打印与解释	6.5h
Day 2 周二 10/13	loss masking 手写实现 + packing 副作用实验	`MASK OK` + 注意力泄漏演示	6h
Day 3 周三 10/14	本地 0.5B LoRA 跑通（信心日）+ 冻结 eval20	adapter 产出 + 基座 20 题基线分	6.5h
Day 4 周四 10/15	上卡：LLaMA-Factory 跑 1.5B LoRA + merge 导出	1.5B adapter + 合并模型 20 题输出	6.5h
Day 5 周五 10/16	灾难性遗忘实测 + replay 缓解 + 打分表定稿	BEFORE-AFTER.md + 三列打分表	6h
Day 6 周六 10/17	LoRA 手推 + rank/alpha 消融 + QLoRA 4bit 冒烟	参数量手算=实测 + 三行消融表	6h
Day 7 周日 10/18	补漏 + 闭卷复测 + 7 项总验收 + 周 7 预习	验收 ≥6/7 项通过	4.5h

注：**本周卡时上限 12 小时，预估花费 ¥25**（4090 按量 ¥1.5-2.5/时，按 ¥2/时算）。分配：Day 3 约 1.5h（0.5B 完整版）、Day 4 约 3.5h（1.5B LoRA + export）、Day 5 约 1h（遗忘实测的推理）、Day 6 约 2h（rank 消融 1.5h + QLoRA 冒烟 0.5h）、机动 4h（重跑/OOM 调试）。**超了怎么办**：先砍 Day 6 的 r=64 那一档（消融只留 8/16），再砍 Day 3 的 0.5B 完整版（降级为「CPU 冒烟 + 卡上 300 步短训」，判据同步改写）。**不许砍** Day 4 的 1.5B 训练和 Day 5 的遗忘实测——它们是 M2 的正文。

注：本周最大风险是**环境版本**而不是算法。`trl` 与 `transformers` 的字段名在大版本间搬过家（例如 `max_seq_length` 挪进 `SFTConfig`），LLaMA-Factory 的 yaml 字段同样随版本变。对策写在「开始前」第 1 步：**装完就把版本钉死并 push**。第二风险是**忘关机**：本周你会频繁开卡跑几十分钟的小实验，一次忘了就是 ¥5-10。

注：砍单顺序（实在做不完按这个砍）：Day 6 任务 3 QLoRA 冒烟（推到周 8 和 7B 一起做）→ Day 2 任务 3 packing（只做「有效 token 占比」那一半，泄漏演示推到 Day 6）→ Day 1 任务 3 的 5 题缩到 3 题。**不许砍**：Day 2 的 loss masking、Day 3/Day 4 的训练、Day 5 的遗忘实测、Day 7 的验收。

二、开始前：环境、模型与数据登记（约 50 分钟，10/11 前一晚做完）

1. 本地 venv 钉版本（Week 5 Day 7 已装，现在补记录）：

```
source E:/study/posttrain/.venv/Scripts/activate
pip install -U transformers datasets accelerate peft trl safetensors modelscope pydantic
pip freeze | grep -Ei "^(torch|transformers|datasets|accelerate|peft|trl|tokenizers)==" \
  > E:/study/posttrain/week06/requirements.pinned.txt
```

`requirements.pinned.txt` 必须 `git push`，**上卡时装同一批版本**（`pip install -r requirements.pinned.txt`）。别在实例上随手 `pip install -U transformers`，那会让本地跑通的代码在卡上报 `unexpected keyword argument`。

2. 下载模型（ModelScope 国内直连比 HF 稳，统一放 `E:/study/models/`）：

```
mkdir -p E:/study/models
modelscope download --model Qwen/Qwen2.5-0.5B          --local_dir E:/study/models/qwen05b
modelscope download --model Qwen/Qwen2.5-0.5B-Instruct --local_dir E:/study/models/qwen05b-instruct
modelscope download --model Qwen/Qwen2.5-1.5B          --local_dir E:/study/models/qwen15b
```

ModelScope 不通就 `export HF_ENDPOINT=https://hf-mirror.com` 后用 `huggingface-cli download Qwen/Qwen2.5-0.5B --local-dir ...`。**只下这三个**，7B（约 15GB）等 Day 6 的 QLoRA 冒烟在卡上下，别在本地占盘。

3. 建目录与登记表：

```
mkdir -p E:/study/posttrain/week06/{day1,day2,day3,day4,day5,day6}
mkdir -p E:/study/llm-posttraining-lab/m2-sft/{data,configs,eval}
```

新建 `m2-sft/data/DATA-SOURCES.md`，**每用一份数据加一行**，字段写死：数据集名 / 获取地址 / **许可（License）** / 下载日期 / 用途 / 是否可再分发。今天先把 Qwen 系列（Apache-2.0）与明天要用的指令集占位写上。

4. AutoDL 余额 ≥¥50。开卡前填 `m2-sft/COSTS.md` 的计划列（表格照周 5），跑完回填实际。

> 注：**数据合规从本周起是硬要求，不是提醒。** 三条规则：① 只用能公开下载、许可明确的数据集，许可页面 URL 记进 `DATA-SOURCES.md`；② `CC-BY-NC` 这类非商业许可的数据只用于你自己的学习实验，**不许混进周 7 要公开发布的数据集**，也不许在简历里写成"我构建并发布了数据集"；③ 真实用户数据（明日记忆的日志）必须匿名化、去掉可识别信息才能进任何数据集——这条到周 7 会变成判据。违反第 ①② 的代价是你公开的东西有法律瑕疵，面试官不懂法，法务懂。

Day 1 • 周一：阶段切换 + 清周 5 gaps + base/instruct 实测 + chat template

今日总览（6.5h）：今天是 Phase 1 与 Phase 2 的分界。**第一个小时用来换脑子**（任务 1，不写代码）。之后清掉从周 5 带来的洞，把 0.5B 的 base 与 instruct 喂同一批问题看差别，最后把 chat template 做到底——它是 SFT 一切 bug 的源头。**数据合规从今天生效**（登记表是任务 2 的硬判据）。今天不开卡，全部本地。

任务 1：心态切换与本周工作准则（约 60 分钟）

1. 读一遍你周 5 写的 `review.md` 第 ⑤ 块（「为什么后训练岗要求会预训练侧的东西」），在 `week06/NOTES.md` 用两句话回答：Phase 1 的产出物和 Phase 2 的产出物差别在哪。
2. 写下本周三条行为准则，之后每个 run 都照做：
   - **一个 run 一行日志，一次只改一个变量**：`run_id，日期，模型，数据版本，lr，rank/alpha，其他改动，eval20总分，卡时，备注`，今天建 `m2-sft/runs.csv` 表头。
   - **每个写进文档的数字必须有出处**：文件名+行号 / 命令 / wandb 链接，三选一。
   - **失败当天入库**：OOM、loss 不降、输出退化，写进 `m2-sft/FAILURES.md`（现象 → 定位 → 处置），这是周 14「失败案例复盘」的原材料。

完成判据：`runs.csv` 表头建好且已有第 0 行（今天的 base vs instruct 就是第一个 run）；三条准则写在 NOTES 顶部；两句话回答写了。

> 注：Phase 1 的产出是「我知道它为什么能 work」，Phase 2 的产出是「我能给出别人能复现的数字」。差别就在日志与对照。今天你觉得多写的这张表麻烦，周 15 项目深挖时面试官问「rank 为什么选 16」，你唯一的武器就是 `runs.csv` 里的三行数字。

任务 2：清周 5 gaps + 指令数据落地与登记（约 90 分钟）

1. 打开 `week05/gaps-from-week5.md`，本周 Day 1-2 只清这三条（动作已排在计划里）：

从上周带来的洞	本周哪天闭合	动作
没接触过「对话格式」的数据	今天任务 4	`apply_chat_template` 打印真实串 + 手写拼接版对比
loss masking 没验证过哪些 token 参与 loss	Day 2 任务 1	3 轮对话样本，断言 `-100` 的 span
微调语境的 lr/warmup 该取多大没概念	Day 3/Day 4	LoRA 用 1e-4 量级，与周 5 从零训的 3e-4~6e-4 记进同一张对照表

2. 取一份中文指令集（**二选一，别都要**）：ModelScope 上的 alpaca-gpt4 中文数据子集，或 HF 镜像上的 Belle 切片。抽 3000 条存 `m2-sft/data/instruct_zh_3k.jsonl`，字段统一为 `instruction` / `input` / `output` 三键（多轮数据周 7 自己造）。
3. 立刻在 `DATA-SOURCES.md` 补一行，许可字段**必须去数据集页面抄原文**，不许写"开源"两个字。

完成判据：三条 gaps 有 `[x]` 或明确落到本周某天；`instruct_zh_3k.jsonl` 行数 ≥2800 且逐行 `json.loads` 无异常（写 5 行校验代码并贴输出）；`DATA-SOURCES.md` ≥3 行且**许可字段无一处为空**——最后这项不满足，今天不许进入训练环节。

任务 3：base 只会续写 vs instruct 会对话（约 120 分钟）

写 `week06/day1/base_vs_instruct.py`，同一批 prompt，**关掉采样（`do_sample=False`）保证可复现**：

```
import json, torch
from transformers import AutoModelForCausalLM, AutoTokenizer

BASE, INSTRUCT = "E:/study/models/qwen05b", "E:/study/models/qwen05b-instruct"
prompts = [
    "把下面这句话翻译成英文：我今天去了图书馆。",
    "用一句话解释什么是学习率。",
    "请列出三个环保习惯，用编号列表输出。",
    "帮我写一封请假邮件，理由是感冒，语气正式。",
    "1 公斤等于多少克？只回答数字和单位。",
]
def gen(model_id, use_template, tag):
    tok = AutoTokenizer.from_pretrained(model_id)
    m = AutoModelForCausalLM.from_pretrained(model_id, torch_dtype=torch.float32).eval()
    rows = []
    for p in prompts:
        msgs = [{"role": "user", "content": p}]
        if use_template:
            s = tok.apply_chat_template(msgs, tokenize=False, add_generation_prompt=True)
        else:
            s = p
        x = tok(s, return_tensors="pt", add_special_tokens=False)
        n_in = x["input_ids"].shape[1]
        with torch.no_grad():
            y = m.generate(**x, max_new_tokens=128, do_sample=False,
                           pad_token_id=tok.eos_token_id)
        rows.append({"prompt": p,
                     "out": tok.decode(y[0][n_in:], skip_special_tokens=True)})
    json.dump(rows, open(f"day1/{tag}.json", "w", encoding="utf-8"), ensure_ascii=False, indent=2)
    print(tag, "OK", len(rows))
gen(BASE, False, "base")            # base 不要套 template，原因见下方注
gen(INSTRUCT, True, "instruct")
```

完成判据：两份 json 生成；`day1/compare.md` 里 5 行对比表（每行：prompt / base 实际行为一句概括 / instruct 实际行为一句概括）；**base 侧至少 2 行出现「续写而非回答」**（典型：接着追问下一个问题、把指令复述一遍、把"请列出三个环保习惯"当成要解释的标题），并贴出你实际看到的原文 2 段。

> 注：**base 模型不要过 chat template。** 模板里的角色标记对 base 是分布外输入，你会把「base 不会对话」误测成「base 输出乱码」，前者是能力问题、后者是分布问题，结论完全不同。想更严谨，多加一列 base + template 的输出做对照，10 分钟的事，面试能讲 3 分钟。另外：CPU 上 0.5B × 5 题 × 128 token 的耗时量级是几分钟，两个模型合起来别超过 20 分钟；明显超了就查 `max_new_tokens` 是否太大、是否真走到贪心分支、模型是否被加载成 float64。

任务 4：chat template 做到底（约 90 分钟）

1. 打印真实字符串与特殊 token（`week06/day1/template.py`）：

```
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained("E:/study/models/qwen05b-instruct")
msgs = [{"role": "user", "content": "你好"},
        {"role": "assistant", "content": "你好，有什么可以帮你？"},
        {"role": "user", "content": "1+1=?"}]
s = tok.apply_chat_template(msgs, tokenize=False, add_generation_prompt=False)
print(repr(s))
print("special:", tok.all_special_tokens)
print("ids head:", tok(s, add_special_tokens=False)["input_ids"][:12])
```

2. 回答三个问题并写进 NOTES：这串文本里哪些是 ChatML 风格的角色标记？`add_generation_prompt=True` 时在末尾多了什么？为什么每轮回答的结束标记必须进 loss（它是"本轮该停了"的唯一监督信号）？
3. **手写拼接对照实验**：用模板字符串自己拼一版同样内容的文本，分别 `encode`，比较 token id 序列；再用它跑一次生成，看输出是否退化。**不许把这版拼接用于训练**。

完成判据：打印出的字符串里能看到成对的角色标记（至少 user/assistant 各一处）；NOTES 三问答全；拼接版与官方版的 id 序列差异位置**逐条列出并解释**（长度差 ≥1 也要能说是哪来的，如换行、BOS、结束标记）。

> 注：`add_special_tokens` 是这周的隐形杀手。`tokenizer(text)` 默认会补 BOS 类特殊 token（各模型策略不同），而 `apply_chat_template` 产出的文本里角色标记已经就位，再补一次就是**双重前缀**，训练时不报错、推理时行为漂移。统一做法：**过模板的文本一律** `add_special_tokens=False`，把这条写进 NOTES 并且贴到仓库 README 的第一屏。

任务 5：收工笔记（约 30 分钟）

NOTES 写：今天最意外的一个现象（base 的原话、或 template 的一处拼接差异）+ 明天开工第一条命令 + 明天几点开始。

今日验收：runs.csv 有第 0 行 + gaps 三条有状态 + DATA-SOURCES 许可无空 + base/instruct 对比表 5 行 + template 三问答全。五项齐才收工。

完成判据：NOTES 三行齐全；`runs.csv` 今日 1 行写明「模型 = 0.5B base / instruct、解码 = 贪心、卡时 = 0」；明天开工第一条命令写进 `day2/todo.md`。

Day 2 • 周二：loss masking 手写实现 + packing 的副作用

今日总览（6h）：今天写的是 SFT 里**唯一会让你掉链子的 20 行代码**：把多轮对话变成 `input_ids` + `labels`，并且只在 assistant 段算 loss。自己手写一遍再去看框架的实现，之后你在 LLaMA-Factory/TRL 里出的任何怪问题都有了排查切入点。今天全部本地 CPU，不开卡。

任务 1：手写 encode + mask（约 150 分钟）

1. 先默写（写不出来再翻知识手册）：**SFT loss 的公式**（只在 assistant 生成段做 next-token 交叉熵）、**prompt 与 user 段的 label 填什么**（`-100`，`F.cross_entropy` 的 `ignore_index` 默认值）、**为什么这么填**（推理时 prompt 是已知条件，让它进 loss 只会教模型"怎么复述问题"，并且会稀释真正的监督信号）。拍照或打字存 `m2-sft/notes/sft-loss.md` ——这份默写 Day 7 闭卷要考。

2. 写 `week06/day2/encode_mask.py`（**前缀增量法**，比按角色标记切字符串稳，因为它不依赖模板长什么样）：

```
import torch
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained("E:/study/models/qwen05b-instruct")
NEG = -100

def encode_messages(msgs):
    ids, labels = [], []
    for i, m in enumerate(msgs):
        # 前缀 = 到第 i 轮为止的全部对话；生成提示仅对最后一轮（assistant）需要
        full = tok(tok.apply_chat_template(msgs[:i+1], tokenize=False,
                                           add_generation_prompt=False),
                   add_special_tokens=False)["input_ids"]
        prefix = tok(tok.apply_chat_template(msgs[:i], tokenize=False,
                                             add_generation_prompt=(m["role"] == "assistant")),
                     add_special_tokens=False)["input_ids"] if i > 0 else []
        add = full[len(prefix):]
        assert full[:len(prefix)] == prefix, "模板不是前缀稳定的：改用整段 tokenize 再定位 span"
        keep = (m["role"] == "assistant")
        ids += add
        labels += (add if keep else [NEG] * len(add))
    return ids, labels

if __name__ == "__main__":
    msgs = [
        {"role": "system",    "content": "你是一个英语老师。"},
        {"role": "user",      "content": "ambitious 什么意思？"},
        {"role": "assistant", "content": "adj. 有雄心的，野心勃勃的。"},
        {"role": "user",      "content": "给一个例句。"},
        {"role": "assistant", "content": "She is ambitious to succeed."},
        {"role": "user",      "content": "它和 aspirational 的区别？"},
        {"role": "assistant", "content": "aspirational 偏'有愿望的'，ambitious 偏'要达成的'。"},
    ]
    ids, labels = encode_messages(msgs)
    x, y = ids[:-1], labels[1:]                    # 右移一位：预测下一个 token
    spans, i = [], 0
    while i < len(y):
        if y[i] != NEG:
            j = i
            while j + 1 < len(y) and y[j + 1] != NEG:
                j += 1
            spans.append((i, j))
            i = j + 1
        else:
            i += 1
    print(f"total tokens: {len(x)} | supervised: {sum(b - a + 1 for a, b in spans)} "
          f"| ratio {sum(b - a + 1 for a, b in spans) / len(x):.2%} | spans: {len(spans)}")
    for a, b in spans:
        print(f"[{a}:{b+1}] {tok.decode(x[a:b+1])!r}")
    print("MASK OK" if len(spans) == 3 and y[0] == NEG else "MASK FAIL")
```

完成判据：脚本打印 `MASK OK`；**打印出的每个 span decode 后与某条 assistant 内容逐字对应**（三段 assistant → 恰好三个 span）；`supervised / total` 比例打印出来并写进 NOTES（预期量级 25%-50%，system 与 user

越长比例越低）。

> 注：右移一位（`x = ids[:-1]`、`y = labels[1:]`）意味着 **assistant 段第一个 token 也是被监督的**——它是"角色标记之后该说什么"，这个信号很重要（学不到就会话模型不认结束标记）。另一处坑：如果打印出 `supervised: 0` 或 `spans` 数量不对，几乎一定是你算前缀的方式与模板输出不一致（比如某次带 `add_generation_prompt=True` 某次不带），上面那条 assert 就是为抓它写的。

任务 2：用真实 CE 验证 mask 生效（约 90 分钟）

1. 把 `encode_messages` 产出的 `labels` 换成「全监督版」（不置 `-100`），用 0.5B base 做一次前向，比较两版 loss 数值差；再取极端对照：`labels` 全填 `-100`，记录行为。
2. 断言 `labels` 出现在词表范围之外也不会崩（`-100` 是 ignore，不是索引）。
3. 把三个数字写进 NOTES：只算 assistant 的 loss、全 token 都算的 loss、全 mask 的结果（**报错还是 nan，记你实测到的那个**）。

```
import torch, torch.nn.functional as F
def ce(logits, labels, neg=-100):
    logits = logits.float()                        # 数值对拍统一 fp32；bf16 下 CE 会漂
    return F.cross_entropy(logits[:, :-1].reshape(-1, logits.shape[-1]),
                           labels[:, 1:], ignore_index=neg)
```

完成判据：打印三行数字且「全 mask」那一行有明确结论（报错信息原文或 nan）；两版 loss 差值 >0 且你能用一句话说清差在哪部分 token 上。

> 注：全 mask 时框架的行为不统一（有的返回 nan，有的抛 `IndexError`，有的返回 0），这没关系——**你要知道自己用的是哪个**。`loss=nan` 会顺梯度污染参数更新，这是「梯度累积里某个 micro-batch 全被 mask」时真实会遇到的事故（周 8 做 DPO 时会再撞一次），今天先记住它长什么样。

任务 3：packing 与它的副作用（约 90 分钟）

1. 统计你这份 3000 条数据的长度分布（token 数 P50/P90/P99），据此定 `max_length`（预期量级：P90 在 300-600，取 1024 作 `cutoff`）。
2. 对比不 packing 与 packing 的**有效 token 数**（去掉 padding 与 `-100` 之后真正参与 loss 的 token 数）：

```
python - <<'PY'
import json, math
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained("E:/study/models/qwen05b")
D = "E:/study/llm-posttraining-lab/m2-sft/data/instruct_zh_3k.jsonl"
rows = [json.loads(l) for l in open(D, encoding="utf-8")][:300]
lens = sorted(len(tok(r["instruction"] + r.get("input", "") + r["output"],
                      add_special_tokens=False)["input_ids"]) for r in rows)
q = lambda p: lens[min(int(p * len(lens)), len(lens) - 1)]
print(f"P50={q(.5)} P90={q(.9)} P99={q(.99)} max={lens[-1]}")

B, L = 8, 1024
valid = sum(lens)
# 不 packing：一个 batch 内所有样本补齐到该 batch 最长的那条
slots_pad = 0
for i in range(0, len(lens), B):
    slots_pad += B * max(lens[i:i + B])
# packing：把样本首尾相接塞满每一行 L
slots_pack = math.ceil(valid / L) * L
e_pad, e_pack = valid / slots_pad, valid / slots_pack
print(f"有效 token 占比  不packing={e_pad:.2%}  packing={e_pack:.2%}  倍数={e_pack/e_pad:.2f}x")
PY
```

3. **注意力泄漏演示**（`week06/day2/leak.py`，CPU 可跑）：把两条不相关样本拼成一条序列，用 `output_attentions=True` 看第二条样本的首 token 是否把注意力分给了第一条样本的 token。

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
M = "E:/study/models/qwen05b"
tok = AutoTokenizer.from_pretrained(M)
m = AutoModelForCausalLM.from_pretrained(M, torch_dtype=torch.float32).eval()
a = "北京的方言属于哪种官话分区？"
b = "用 Python 写一个二分查找。"
ida, idb = tok(a, add_special_tokens=False)["input_ids"], tok(b, add_special_tokens=False)["input_ids"]
seq = torch.tensor([ida + idb])
with torch.no_grad():
    out = m(seq, output_attentions=True)
attn = out.attentions[0][0]                          # (heads, T, T)，取第 0 层第 0 头
row = attn[len(ida), :len(ida)]                      # b 的首 token 对 a 的各 token
print(f"|a|={len(ida)} |b|={len(idb)} 泄漏权重合计={row.sum():.4f} 最大={row.max():.4f}")
```

完成判据（两个占比的分子都是「参与 loss 的 token 数」，这里简化成全部 token，严格版用任务 1 的函数减掉 `-100`）：长度分布三个分位数入库；脚本打印的 `倍数 ≥1.3`（预期量级 1.5-4，取决于长度分布，**以你的实测为准**）；泄漏演示打印的「泄漏权重合计 >0」——**这就证明了朴素拼接会让第二条样本看到第一条**，NOTES 写清解法：packing 必须配合 position_ids 每段重置 + 段间块状（block-diagonal）attention mask，FlashAttention 的 varlen 路径就是干这个的。

> 注：这一条面试官爱问："你开 packing 遇到过什么坑？" 只会说"提升吞吐"的是没做过的人。能说出"注意力会串到上一条样本，所以要 varlen / block-diagonal mask，不然模型学到跨样本依赖"才算真懂，而这个演示只花你 20 分钟。

任务 4：闭卷默写收口 + 收工（约 30 分钟）

合上所有文件，在 `m2-sft/notes/sft-loss.md` 补两行：SFT loss 公式、mask 规则（哪些 role 的哪些 token 参与 loss、右移几位）。这是 Day 7 的考题，现在先自己判一次，错了明天开始每天早晨重默一遍（1 分钟）。

今日验收：`MASK OK` + span 逐字对应 + 三行 loss 数字 + packing 两项达标 + 默写文件存在。

完成判据：`m2-sft/notes/sft-loss.md` 两处（公式、mask 规则）非空且不含「差不多/大概」这类含糊词；NOTES 新增 ≥3 条。

Day 3 • 周三：本地 0.5B LoRA 跑通（信心日）+ 冻结 eval20

今日总览（6.5h）：今天你要跑出**第一个属于自己的 SFT 模型**（虽然只有 0.5B），所以刻意先在本地 CPU 建立信心，再花 1.5h 卡时把完整训练跑完。同时冻结本周和之后三周都要用的 20 题评测集——**它一旦定了就不许改**（改了前后分数不可比，这是评测的第一纪律）。

任务 1：CPU 冒烟：TRL SFTTrainer + LoRA（约 120 分钟）

`week06/day3/sft_tiny.py`：

```
import torch
from datasets import load_dataset
from peft import LoraConfig
from transformers import AutoModelForCausalLM, AutoTokenizer
from trl import SFTConfig, SFTTrainer

M = "E:/study/models/qwen05b"
tok = AutoTokenizer.from_pretrained(M)
model = AutoModelForCausalLM.from_pretrained(M, torch_dtype=torch.float32)   # CPU 用 fp32
model.config.use_cache = False
ds = load_dataset("json", data_files="E:/study/llm-posttraining-lab/m2-sft/data/instruct_zh_3k.jsonl")
["train"]

peft = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                  target_modules=["q_proj", "k_proj", "v_proj", "o_proj"], bias="none",
                  task_type="CAUSAL_LM")
args = SFTConfig(output_dir="E:/study/posttrain/week06/day3/smoke-lora",
                 max_length=512, per_device_train_batch_size=2, gradient_accumulation_steps=4,
                 learning_rate=1e-4, num_train_epochs=1, max_steps=20,
                 logging_steps=2, save_strategy="no", report_to="none",
                 seed=42, use_cpu=True)
tr = SFTTrainer(model=model, args=args, train_dataset=ds.select(range(500)),
                peft_config=peft, processing_class=tok)
tr.train()
tr.save_model("E:/study/posttrain/week06/day3/smoke-lora")
print("CPU SMOKE OK")
```

完成判据：打印 `CPU SMOKE OK`；20 步里 loss 有下降趋势（**不要求收敛**，要求"末步 < 首步"，写清两个数字）；`smoke-lora/` 下有 `adapter_config.json` 与 `adapter_model.safetensors`；`model.print_trainable_parameters()`

打印出的可训参数量记进 NOTES。

> 注：这段代码里最容易报错的是 **TRL 参数名**。如果报 `SFTConfig.__init__() got an unexpected keyword argument 'max_seq_length'`，说明你装的版本已改名为 `max_length`（或反过来）—— `python -c "import inspect,trl; print([p for p in inspect.signature(trl.SFTConfig).parameters if 'len' in p or 'step' in p])"` 一行看清，别猜也别搜半小时。CPU 冒烟 20 步耗时量级 5-15 分钟，超过 30 分钟就查是不是每条都过了 tokenizer（`SFTTrainer` 会对整个 dataset 预处理，500 条不该慢，慢就是 `max_length` 太大或数据集没截断）。

任务 2：上卡跑 0.5B LoRA 完整版（约 120 分钟，卡时 1.5h）

1. 开卡（4090），`pip install -r requirements.pinned.txt`，把 `E:/study/models/qwen05b` 与数据用 AutoDL 的 JupyterLab 上传或从 ModelScope 直接在卡上下（后者更快，别传 1GB 文件）。
2. 改三处：`device`（去掉 `use_cpu`）、`max_steps=-1`、`num_train_epochs=1`、数据全量 3000 条、`bf16=True`。启动前先跑 30 步确认无 nan。
3. 训练期间做任务 3（这是本周教你的习惯：**等待时间必须产出别的交付物**）。

完成判据：train loss 末值 < 首值的 **50%**（预期量级 1.1-1.4 → 0.5-0.7，**以你实测为准写进 runs.csv，别照抄这里的数**）；`eval_loss` 未出现上翘（若上翘，写一句你打算怎么改）；adapter 落地并 `git push` 记录配置（**adapter 权重本身传网盘或留在持久盘，仓库里只放路径与 sha256**）。

> 注：0.5B + LoRA 在 4090 上 3000 条 1 epoch 的耗时量级是十几到几十分钟（取决于 `max_length` 与 batch）。若你 1.5h 还没跑完，说明 batch/序列长度设置不合理（`max_length=512`、等效 batch 32 是今天的基准），先降 batch 再等，别加大。

任务 3：冻结 eval20（自建 20 题，禁止抄公开评测集）（约 120 分钟）

1. 自己出 20 道题，四类各 5 道：通用常识问答 / 指令跟随与格式约束 / 中英互译 / 教学场景解释（最后一类是周 7、周 12 的种子）。存 `m2-sft/eval/eval20.jsonl`，字段 `id, category, question, reference_points`。
2. 写 0-2 分 rubric（**必须写死三档，别写"看感觉"**）：
   - 0 分：答非所问、事实错误、或未按格式要求；
   - 1 分：方向对但漏要点 / 格式部分不符 / 有明显多余套话；
   - 2 分：要点齐全、格式严格符合、无多余内容。
3. 打分脚本 `m2-sft/eval/score.py`：**遮住模型名**（盲评），每题打分必须写一句理由。
4. 先给**基座 0.5B**（base）与**官方 instruct 0.5B** 各打一遍 20 题（贪心解码，temperature 写死 0），产出两张表。

完成判据：`eval20.jsonl` 恰好 20 行、四类各 5、`git add` 后打上 tag `eval20-v1`（**从此不许改**，要改只能新增 `eval21`）；rubric 三档定义写清；两张打分表合计 40 行、每行有分数与理由。

> 注：**为什么必须自建而不能用现成 benchmark 的题？** 两个理由：① 评测集不能以任何形式进训练集（周 7 的去污染会正式处理这条），而公开 benchmark 你没法保证你的 base 模型没见过——它多半见过；② 面试时"我用 lm-eval 跑了 GSM8K"人人都会说，"我有一组 20 题的自建回归测试、三周不改动只对比分数"才是训练工程师的口吻。

任务 4：收工记账（约 30 分钟）

回填 `COSTS.md`（今天计划 1.5h）；`runs.csv` 加两行（CPU 冒烟 + 卡上完整版）；关机确认。

今日验收：CPU SMOKE OK + 卡上 loss 末值 < 首值 50% + eval20 冻结打 tag + 基座与 instruct 两张打分表。

完成判据：`COSTS.md` 今日行填满（计划 1.5h / 实际 ?h）；`runs.csv` 两行各自带 loss 数字；控制台显示「已关机」。

Day 4 • 周四：上卡用 LLaMA-Factory 跑 1.5B LoRA + merge 导出

今日总览（6.5h）：今天是本周的主菜：**用业界主流框架而不是自己写的脚本，训一个 1.5B 的 LoRA**，并把 adapter merge 进基座做出一个可独立加载的模型。为什么用框架？后训练团队日常就是它，你要练的是"读配置、改字段、看日志、排报错"，而不是重复造轮子。卡时预算 3.5h。

任务 1：装 LLaMA-Factory 并注册数据集（约 90 分钟）

```
cd /root/autodl-tmp
source /etc/network_turbo
git clone https://github.com/hiyouga/LLaMA-Factory.git
cd LLaMA-Factory && pip install -e ".[torch,metrics]" -q
llamafactory-cli version
mkdir -p data && cp E:/study/llm-posttraining-lab/m2-sft/data/instruct_zh_3k.jsonl data/   # 实际用 scp 或卡上重新下载
```

1. 打开仓库里的 `data/dataset_info.json`，照已有条目的格式注册你的数据（列名映射写清 `instruction/input/output` 对应哪些 key）。
2. 打开 `examples/train_lora/` 下与 Qwen 对应的 yaml 作为模板，**逐字段过一遍**（模型路径、`stage: sft`、`finetuning_type: lora`、`template`、`cutoff_len`、`learning_rate`、`lora_rank`、`lora_alpha`、`per_device_train_batch_size`、`gradient_accumulation_steps`、`preprocessing_num_workers`），存成 `m2-sft/configs/qwen15_lora.yaml`。

完成判据：`llamafactory-cli version` 有输出（无输出=装失败，去查 requirements 版本冲突，限时 30 分钟，超时就改用 TRL 版本跑今天的训练并把这件事记进 FAILURES）；`dataset_info.json` 里你的条目能被 `llamafactory-cli train <yaml>` 认出（**报错信息里出现你的数据集名 = 识别成功**）。

> 注：`template` 字段必须填对（Qwen 系列填 `qwen`）。填错的后果是训练照跑、loss 照降、模型输出混乱——**这是 SFT 最贵的一类 bug，因为没有任何报错**。你 Day 1 手写模板的那 90 分钟，就是为了在这种时刻能一眼看出"输出里角色标记不对"。

任务 2：1.5B LoRA 训练（约 180 分钟，卡时约 3h 含重试）

基准配置（**这些是社区常用量级，不是你今天的实验结论；跑完把实测写进 runs.csv**）：`lora_rank=16`、`lora_alpha=32`、`target=all`（框架的写法）、lr `1e-4`、`cutoff_len=1024`、`per_device_train_batch_size=4`、`gradient_accumulation_steps=8`（等效 batch 32）、1 epoch、`bf16`、`gradient_checkpointing=True`、warmup ratio 0.03。

```
export WANDB_MODE=online        # 连不上改 offline，回家 wandb sync
FORCE_TORCHRUN=1 llamafactory-cli train m2-sft/configs/qwen15_lora.yaml 2>&1 | tee /root/autodl-tmp/lora15.log
```

完成判据：训练日志里 train loss 首末两个数（预期量级 1.0-1.4 → 0.5-0.8，**以实测为准**）；`epoch` 跑完 1；输出目录含 `adapter_config.json`、`adapter_model.safetensors`、`trainer_state.json`；峰值显存打印（`nvidia-smi --query-gpu=memory.used --format=csv` 采 3 次取最大，写进 COSTS.md 旁边）。

> 注：**先 1.5B 跑通再谈别的规模**，这是记账纪律。4090 24G 上 1.5B + LoRA(bf16) 应当很舒适；如果 OOM，按顺序查：`cutoff_len` 是不是被设成 4096（序列翻倍激活翻倍、attention 四倍）→ batch 是不是 4 太大 → `gradient_checkpointing` 有没有生效 → `preprocessing_num_workers` 无关但常被人误改。**注意 1.5B 全量微调在 4090 上是很紧的**（bf16 下 AdamW 优化器态就要 ≈18GB），这就是计划排 LoRA 在前、全量放到以后的原因；本周不做 1.5B 全量，别临时加需求。

任务 3：merge adapter 并验证合并等价（约 60 分钟）

1. 用框架的 export（`llamafactory-cli export <合并用 yaml>`）把 adapter 合并进基座，或自己用 peft：

```
from peft import PeftModel
from transformers import AutoModelForCausalLM, AutoTokenizer
base = "E:/study/models/qwen15b"          # 卡上路径
adapter = "/root/autodl-tmp/out/qwen15-lora"
tok = AutoTokenizer.from_pretrained(base)
m = AutoModelForCausalLM.from_pretrained(base, torch_dtype=torch.bfloat16)
m = PeftModel.from_pretrained(m, adapter).merge_and_unload()
print("merged params:", sum(p.numel() for p in m.parameters()))
```

2. **等价性抽查**：同一条 prompt（固定 seed、贪心），分别用「基座 + adapter（不 merge）」与「merge 后的完整模型」生成 64 token，比较输出文本与首步 logits。

完成判据：merge 成功且模型目录能单独 `from_pretrained` 加载（不依赖 adapter 路径）；**两个版本对同一条 prompt 的贪心输出完全一致**（若不一致，差异要能解释：浮点/合并顺序/温度）。NOTES 写两句 merge 的实际作用（推理时把 `BA` 加回 `W`，因此线上不需要额外算子）——**这正是「LoRA 上线/回滚成本低」那道面试题的答案**。

任务 4：跑 20 题（基线补齐）（约 30 分钟，卡时约 0.5h）

用 merge 后的模型跑 `eval20.jsonl`，贪心解码，输出存 `m2-sft/eval/pred-15b-lora.jsonl`。打分留到明天（今天只收输出，别一边看结果一边改模型，那是评测的大忌）。

完成判据：20 条预测齐全（jsonl 20 行、含模型名与解码参数），无一条是空串或纯角色标记残留。

任务 5：记账与收工（约 30 分钟）

回填 COSTS（今天计划 3.5h，累计不许超 8h）；`runs.csv` 至少 3 行；关机确认。

今日验收：yaml 跑通 + loss 首末数入库 + merge 等价抽查通过 + 20 题预测入库 + 已关机。

完成判据：`COSTS.md` 本周累计 ≤6h（含 Day 3 的 1.5h）；新增的 OOM/报错在 `FAILURES.md` 里有「现象 → 定位 → 处置」三行；已关机。

Day 5 • 周五：灾难性遗忘实测 + replay 缓解 + 打分表定稿

今日总览（6h）：SFT 会让模型更会对话，也会让它忘事。今天你要用数字回答两个问题：**忘了多少**（基座 vs SFT 后在通用问答上的差），**加一点 replay 能不能补回来**。这是后训练岗日常里最真实的一天：你 80% 的时间在评测与数据配比，不在调 lr。卡时 1h（本地 CPU 也能跑，但 1.5B 在 CPU 上太慢，所以今天开卡做推理）。

任务 1：遗忘实测（约 120 分钟，卡时 1h）

1. 从 `eval20.jsonl` **之外**另出 10 道通识题（周 6 之前 base 就会答的：单位换算、简单算术、常识解释、成语释义），存 `m2-sft/eval/forget10.jsonl`。理由写清：**SFT 题和遗忘题必须是两套**，否则你分不清"没学会"和"忘了"。
2. 三个模型各跑 10 题 + 20 题（贪心、`max_new_tokens=128`）：Qwen2.5-1.5B（base）、1.5B+LoRA（昨天）、官方 Qwen2.5-1.5B-Instruct（**上限参照**；本地没下就今天顺手在卡上下，约 3GB，只用于对比不用于训练）。
3. 打分用 Day 3 的 rubric（0-2），三列并排。

完成判据：`m2-sft/BEFORE-AFTER.md` 里有 10 行遗忘对比表 + 20 行指令对比表，**至少 1 处明确退化**（SFT 后分数低于 base 的题），并贴出退化输出原文；若一处都没有，加测 10 道更"通用"的题（比如让它写代码、算多步数学），并在文档里诚实写"本轮未观察到遗忘，可能因为数据量小、epoch=1"——**未观察到也是结论，不许编**。

> 注：为什么 1 epoch、3000 条也可能看不出遗忘？因为 LoRA 冻结了主干、可训参数才百分之几，天然抗遗忘——**这本身就是面试答案**（"LoRA 缓解灾难性遗忘的原因：主参数不动，改动被限制在低秩增量里"）。把它写在表格下面，比只报一个分数值钱。

任务 2：replay 缓解实验（约 90 分钟，本地 CPU 或 0.5B 短卡）

1. 构造混合数据：`instruct_zh_3k.jsonl` + **10% 的"续写型"样本**（把周 5 留下的莎士比亚/TinyStories 文本按 LM 目标转成对话形态，或直接混入未 mask 的续写样本），得到 `instruct_replay_mix.jsonl`。
2. 用 0.5B + LoRA 各跑一次短训练（500 步，同 lr 同 batch，唯一变量是数据）：纯 instruct vs replay 混合。
3. 同一批 forget10 题对比两者。

完成判据：runs.csv 里两行（唯一差异列为"数据"）；退化题在 replay 版恢复 **≥1 题**（分数回升 ≥1 分）。若没恢复：把「replay 比例 10% 可能不够」写进 gaps 并标到周 7（数据工程周会正式处理配比），不许静默放弃这个实验。

> 注：replay 的比例、混合方式在业界没有定论，你今天的目的是拿到"我做过配比实验"这件事本身和一组真实数字，不是找到最优解。别在这上面调 5 组超参——那是烧卡时的黑洞（今天的卡时上限就是 0.5h，超了就改到 CPU 上跑 0.5B 的 100 步版本）。

任务 3：三列打分表与 rubric 定稿（约 90 分钟）

把 `m2-sft/eval/score-table.md` 定稿：行是 20 题（+10 遗忘题），列是 base / 0.5B-LoRA / 1.5B-LoRA / 官方 instruct，分数 + 每题一句理由。做完再检查两件事：① 总分是否单调合理（官方 instruct 通常最高，你的 1.5B 应高于 base）；② **抽 3 题重打一遍（隔 20 分钟、遮住模型名）**，看自己给的分是否一致。

完成判据：表格 30 行 × 4 列齐全、每格有分数；一致性抽查 3 题里 **≥2 题分数完全一致**（不一致就回去把 rubric 的某档定义收紧，这是评测标定，不是作弊）。

> 注：这一条"自己和自己的一致性"是 LLM-as-judge 与人评共同的问题，周 12 要用温度=0 + 重跑 3 次来处理。你今天做的 3 题抽查就是那个协议的最小版本，先养成习惯。

任务 4：把今天的东西写进文档（约 60 分钟）

`m2-sft/BEFORE-AFTER.md` 结构定死：结论三行 → 两张表 → 一处退化原文 → replay 实验数字 → 没解决的三个问题。NOTES 加 ≥5 条。

今日验收：遗忘表 + 退化证据 + replay 结论（含"未恢复"的诚实版）+ 四列打分表 + 一致性抽查数字。

完成判据：`BEFORE-AFTER.md` 五块齐全（结论三行 / 两张表 / 退化原文 / replay 数字 / 三个未解问题），每个数字后面跟文件路径或命令。

Day 6 • 周六：LoRA 深挖（手推 + 参数量 + rank/alpha 消融）+ QLoRA 冒烟

今日总览（6h）：今天把 PEFT 从"会用"推到"会解释 + 会选"。上午手推 ΔW=BA 并**算出可训参数量和实测对上**（这个对账习惯是训练工程师的基本功），中午做三档 rank 的小消融，下午用 4bit 加载 7B 跑 100 步（QLoRA 的体感）。卡时 2h。

任务 1：LoRA 手推与参数量对账（约 90 分钟）

1. 默写：`ΔW = BA`，`B ∈ (r × d_out)`、`A ∈ (r × d_in)`，前向是 `h = Wx + (α/r)·B A x`；初始化是 **A 随机、B 置零**（为什么？——保证训练起点等价于原模型，不破坏预训练权重；若两边都随机，起点就被污染了）。写进 `m2-sft/notes/lora.md`。

2. **手算可训参数量**：1.5B 的 hidden size 与层数从 `config.json` 读（`hidden_size` / `num_hidden_layers` / `num_key_value_heads`），对 `target_modules = [q_proj,k_proj,v_proj,o_proj]` 且 `r=16`，算 `Σ (d_in + d_out) · r`；再算全量微调的参数量做比。
3. 和 `model.print_trainable_parameters()` 的输出对账。

```
import json
cfg = json.load(open("E:/study/models/qwen15b/config.json"))
print({k: cfg[k] for k in ["hidden_size","num_hidden_layers","num_attention_heads",
                           "num_key_value_heads","intermediate_size"] if k in cfg})
```

完成判据：手算值与框架实测值差 **<1%**（差得多必有具体原因：注意 K/V 用了 GQA 所以 `num_key_value_heads` 更小，`k_proj` 的输出维不是 hidden_size——**这就是面试官爱挖的那一层**）；NOTES 写清 α 与 r 的比值为什么重要（有效学习率 `α/r`，所以 r=8/α=16 与 r=64/α=128 是"同倍率不同容量"）。

任务 2：rank/alpha 消融（约 90 分钟，卡时 1.5h）

三档跑同一份数据、同一 lr，只改 rank/α（α=2r 保持倍率）：`r=4/α=8`、`r=16/α=32`、`r=64/α=128`，每档 300 步短训 + eval20 里的 6 题子集（固定题号，贪心）。预算紧就砍掉 r=64（本周允许，写进 COSTS）。

完成判据：三行表（rank / 可训参数量 / 300 步后 train loss / 6 题子集分 / 峰值显存）；能写出**一句不成立的直觉**并被数据推翻（如"rank 越大 loss 一定越低"或"参数量与显存成正比"——LoRA 的显存大头是基座与激活，不是可训参数，实测会教你这一点）。

> 注：短训 300 步的分数差异非常容易被随机性吃掉。为了让消融有意义，三档必须**同一个 seed、同一批数据顺序**，并且在报告里写"步数太少，只用于验证管线与量级，不作为选 rank 的最终依据"。**诚实的实验注释比一个好看的结论更有面试价值。**

任务 3：QLoRA（NF4）冒烟（约 90 分钟，卡时 0.5h）

```
pip install -U bitsandbytes accelerate
```

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model
bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                         bnb_4bit_compute_dtype=torch.bfloat16,
                         bnb_4bit_use_double_quant=True)
M = "Qwen/Qwen2.5-7B"          # 卡上从 ModelScope 下到持久盘再指本地路径
tok = AutoTokenizer.from_pretrained(M)
m = AutoModelForCausalLM.from_pretrained(M, quantization_config=bnb, device_map="cuda:0")
m = get_peft_model(m, LoraConfig(r=16, lora_alpha=32, task_type="CAUSAL_LM",
                                 target_modules=["q_proj","k_proj","v_proj","o_proj"]))
m.print_trainable_parameters()
x = tok("用一句话解释 NF4 量化。", return_tensors="pt").to(m.device)
out = m(**x, labels=x["input_ids"])
print("QLoRA forward OK", float(out.loss))
```

完成判据：模型加载成功、打印出的 7B 可训参数占比（预期量级 <0.5%）、`nvidia-smi` 峰值显存 <24GB、前向 loss 是有限数（非 nan/inf）。**加载失败或 OOM 的降级路线按顺序走**：① 换 1.5B 走同样代码（验证 4bit 通路）；② 加 `paged_adamw_8bit` 优化器；③ 本周放弃、写进 gaps 推到周 8（7B 的 QLoRA 在 DPO 周本来还要用一次，那时环境通常已修好）。

> 注：为什么 QLoRA 能省这么多？基座权重以 NF4 存（≈0.5 字节/参，bf16 是 2 字节）且**冻结**，LoRA 参数仍是 bf16 参与梯度；反向时按 block 反量化再算。paged optimizer 是为对付显存碎片造成的瞬时尖峰。这段在面试里能讲 3 分钟，今天你亲手跑过就是有据可讲。

任务 4：README 与诚信声明（约 60 分钟）

`m2-sft/README.md` 写四段：① 这是什么（1.5B LoRA + 20 题回归评测）② **哪些是我自己写的（`encode_mask.py`、packing 演示、eval20、打分脚本、消融配置），哪些是框架做的（LLaMA-Factory/TRL 的训练循环）** ③ 怎么复现（三条命令 + 依赖 pinned 文件）④ 结果与失败（loss 数字、eval20 总分、一处退化）。

完成判据：第 ② 段存在且逐条列（不许写"部分自己实现"这种含糊话）；README 里每个数字都有对应文件路径。

任务 5：收工记账（约 30 分钟）

累计卡时对照 12h 上限；超了就砍明天预习里的加分项，不许砍验收。

今日验收：参数量手算误差 <1% + 三行（或两行）消融表 + QLoRA 冒烟 OK 或有明确降级记录 + README 四段齐全。

完成判据：本周累计卡时 ≤12h（超了就按砍单顺序砍掉消融第三档并写明）；消融行（3 行或 2 行）进 `runs.csv`；已关机。

Day 7 • 周日：补漏 + 闭卷复测 + 本周总验收 + 周 7 预习

今日总览（4.5h）：不开卡。今天做四件固定动作 + 一次本周账单核对。晚上把 `m2-sft` 打成 tag `week06-m2-start`，它以后是周 8 DPO 的起点模型。

任务 1：补漏（约 90 分钟）

打开 `gaps.md` 与 `FAILURES.md`，**最多挑 2 个洞**补（贪多必失）。优先级：闭卷要考的 mask/LoRA 细节 > 没跑成的 QLoRA > 评测表里不一致的题。

完成判据：最多 2 个洞，每个留一条证据（重跑命令 + 数字，或重默拍照入库）；`FAILURES.md` 对应条目状态更新。

任务 2：闭卷复测（约 60 分钟）

新文件 `week07/test-closedbook.md`，限时 30 分钟写完，再对答案 30 分钟：

1. 写出 SFT 的损失函数（含求和范围），并用文字说明 `label=-100` 出现在哪些位置、为什么右移一位后 assistant 段首 token 仍参与 loss。
2. 给一段 3 轮对话（system+user+assistant ×2），手写它的 labels 掩码边界（画出 span 起止，不必写代码）。
3. 默写 LoRA 前向公式与 α/r 的作用；说清 A/B 的初始化及原因。
4. 一句话答：为什么 base 模型不能直接用 chat template 的输出做对比实验？packing 需要注意什么？

完成判据：**4/4 题写满**且第 1、2 题的关键点（求和范围只在 assistant 段、`-100` 位置、右移、span 边界）**全对**；错的点当场写进 gaps 并排到周 7 Day 7 复测。

任务 3：本周总验收（约 40 分钟）

#	验收项	判据
1	chat template	打印过真实串、特殊 token 与角色标记；手写拼接版差异逐条解释；NOTES 里有「过模板必 `add_special_tokens=False`」
2	loss masking	自写脚本打印 `MASK OK`，三段 assistant span 与原文逐字对应；监督 token 占比有数字
3	packing	有效 token 比 ≥1.3×；跨样本注意力泄漏有实测数值与解法一句
4	0.5B LoRA	CPU 冒烟 `CPU SMOKE OK` + 卡上 train loss 末值 < 首值 50% + adapter 落地
5	1.5B LoRA + merge	LLaMA-Factory 跑通 1 epoch；merge 后模型可独立加载；贪心输出与未 merge 版一致
6	评测体系	`eval20.jsonl` 打 tag 冻结、四类各 5；四列打分表齐全 + 一致性抽查 ≥2/3；遗忘退化有证据或有诚实"未观察到"
7	PEFT 深度	可训参数手算与实测差 <1%；rank/α 消融 ≥2 行；QLoRA 冒烟有结果或有降级记录

≥6 项通过 = 本周合格（M2 起步达成）；不足 6 项，缺的写进周 7 Day 1 的前 90 分钟，并注明用什么动作补。

完成判据：表格 7 行都有勾选或缺项补齐日期；`m2-sft` 已打 tag `week06-m2-start`（周 8 的起点模型）。

任务 4：新暴露的洞 → 后续周映射（约 40 分钟）

`week07/gaps-from-week6.md`，格式沿用「错了什么 → 哪个空洞 → 补在哪天 → 什么动作」。**本周至少 4 条，且必须落到周 7/周 8**：

洞	归到	动作
只用过别人清洗好的指令集，没做过数据生产与清洗	周 7 Day 2-4	自建生成 + 四板斧过滤管线（去重/困惑度/RM/规则）

洞	归到	动作
多轮对话数据没造过（今天是单轮 alpaca 形态）	周 7 Day 1/Day 2	按 5 类教学任务设计多轮 schema 并生成
不知道合成数据的提示词该怎么版本化与复现	周 7 Day 2	生成脚本里固定 seed/温度 + 提示版本号进文件名
没做过 RM 打分与偏好数据	周 8 Day 1-2	用周 6 的模型输出造偏好对（同 prompt 采 3-4 个 → 打分 → max-min 配对）
评测集只有 20 题，统计功效不足	周 7（扩到教学场景 50 题）→ 周 12（200 题 + LLM-as-judge 一致性）	从周 7 数据集里切 held-out 评测子集并做去污染

完成判据：表 ≥4 行且每行有动作；`eval20` 与 `m2-sft` 已打 tag；`runs.csv` 全周 ≥10 行。

任务 5：周 7 预习 + 本周对账（约 40 分钟）

1. 读 self-instruct 与 evol-instruct 的方法部分（各 10 分钟），写 5 行：两者分别解决"指令太少"和"指令太像"的哪个？
2. 打开你周 6 的 `instruct_zh_3k.jsonl`，挑 10 条最像的（同一种句式），并排贴进 `week07/preview.md`，写一句话回答："如果你来造数据，怎么让第 11 条不像前 10 条？"
3. 想清楚周 7 的选题：**从明日记忆的真实查词/教学场景合成"英语教学对话"指令数据并发布**。本周只是预告，不要动手。
4. `COSTS.md` 汇总：计划 12h / 实际 ?h；计划 ¥25 / 实际 ?。时间真实花销按天记（对照 42h），任何超支写一句原因（多半是环境重装或反复重跑同一实验）。

完成判据：5 行笔记 + 10 条相似样本与那句话回答 + COSTS 的实际列全部填完。

今日验收：验收表 ≥6 项打勾 + 闭卷 4/4 + gaps 表 ≥4 行 + 账单核对完成。

三、常见问题与纪律

- **loss 降不下去怎么办？** 你已经会在周 5 学的顺序排查，这里照用：**lr（LoRA 常用 1e-4 量级，从零训 LM 是 3e-4~6e-4，混了就崩）→ 数值精度（bf16 微调是稳的，fp16 需要 scaling）→ 数据（模板错了、mask 全 0、字段错位）→ 初始化（LoRA 的 B 必须为 0）**。数据那一档还要多加一条 SFT 特有的：**mask 检查**——今天你手写过，所以能查。
- **框架跑不通怎么办？** 限时 30 分钟。超时就**降级到 TRL 版**（Day 3 的脚本改模型路径就能跑 1.5B），把框架的报错原样写进 `FAILURES.md`，周 8 再战。别把整个 Day 4 献给一个 yaml。
- **卡时超了？** 立刻停手回填 COSTS，然后按总览的砍单顺序砍。本周允许"1.5B LoRA 跑 0.5 epoch"作为降级交付，**不允许**取消 Day 5 的遗忘实测（那是 M2 里唯一能讲出"我懂配比"的东西）。
- **数据合规复查**：本周结束前打开 `DATA-SOURCES.md`，逐行确认：许可字段是页面原文而非你的推测；非商业许可的数据没有出现在任何你打算公开的地方（周 7 发布的数据集、博客贴图、repo 里的 jsonl）。有一处不清楚就去把页面抄下来，别猜。

- **别提前跳 DPO/RLHF。** 周 8 才做偏好。现在跑去调 `DPOTrainer`，你会遇到"loss 是负的"（DPO loss 可以为负）这种需要 Day 2 那种 masking 功力才能看懂的现象，浪费一周。
- **AI 可以做什么**：解释报错、帮你看 diff、给 rubric 的措辞提建议。**不能做**：替你写 eval20 的题、替你打分、替你编 runs.csv 里的数字。面试追问三层就会塌。